Lesson notes · for you

Searching

Programming · Level 3 · Lesson 3

Time
20 min

Materials
A phone with this lesson downloaded; A paper dictionary or a sorted register (optional)

Learning goals

- Write linear search.
- Explain binary search, and why the list must be sorted.
- Count the worst-case looks for 1,000 items.

Activity: Find the card

1. Warm-up (3 min): A partner lays 16 numbered cards face down in a random order. Find their chosen number by turning cards over one at a time. Count the turns.
2. Sorted (7 min): Now lay the cards in order. Turn over the middle card, then throw away the half that can't hold the number. Repeat.
3. Count (7 min): Do each way three times and write down the turns. Which never needs more than 5?
4. Explain (3 min): Shuffle the cards and try halving. What goes wrong?

Playing cards or numbered paper squares both work. No phone is needed until the code.

Discussion

- Why does binary search need a sorted list?
- When is linear search good enough?
- Why does a list a thousand times bigger need only about 10 more looks?

Check yourself

- How many looks for 1,000 items: linear worst case, and binary worst case?

Try it this week
Play "guess my number" from 1 to 1,000 with a friend using halving. Can you always do it in 10 guesses?

Worksheet

Searching

Programming · Level 3 · Lesson 3

Name _____ Class _____ Date _____

1 **A sorted list has 1,024 names. At most, how many looks does binary search need?**

◯ 1,024 ◯ 11 ◯ 512

2 **Write find(names, target): linear search. It returns the position of the first match, or -1 if the target isn't there.**

```
def find(names, target):
    # position of target, or -1
    return -1
```

Write your program:

3 **Put binary search in order. lo and hi mark the part of the sorted list still possible. One line doesn't belong.**

Number the lines in order (1, 2, 3…), and draw one arrow → in front of a line for each indent it needs. Cross out any line that doesn't belong.

☐ `mid = (lo + hi) // 2`

☐ `if a[mid] == x:`

☐ `while lo < hi:`

☐ `return mid`

☐ `elif a[mid] < x:`

☐ `def search(a, x):`

☐ `hi = mid - 1`

☐ `else:`

☐ `lo, hi = 0, len(a) - 1`

☐ `lo = mid + 1`

☐ `while lo <= hi:`

☐ `return -1`

4 **What does this program print?**

```
names = ["Ada", "Kofi", "Layla", "Omar"]
print(names.index("Omar"))
```

◯ 3 ◯ Omar ◯ 4

5 **Which are true about binary search?**

☐ Each look halves what is left ☐ It works on any list, sorted or not ☐ 1,000 items need about 10 looks at most

☐ The list must be sorted

✎ **Where in your life do you search a sorted list (a dictionary, contacts, a register)? How do you search it: one by one, or by halving?**

Check your answers

Searching

Programming · Level 3 · Lesson 3

1 **11**

Halving 1,024 reaches 1 after 10 halvings, plus the final look at the last name: 11.

2

```
def find(names, target):
    for i in range(len(names)):
        if names[i] == target:
            return i
    return -1
```

Return the position the moment you find the target; return -1 only after the loop has checked every item.

3 **Top to bottom: 4, 5, X, 6, 7, 1, 10, 9, 2, 8, 3, 11**

```
def search(a, x):
    lo, hi = 0, len(a) - 1
    while lo <= hi:
        mid = (lo + hi) // 2
        if a[mid] == x:
            return mid
        elif a[mid] < x:
            lo = mid + 1
        else:
            hi = mid - 1
    return -1
```

Look at the middle; if the target is bigger, move lo past mid, else move hi below it.

4 **3**

index() is a linear search: it checks from the start and returns the position, counting from 0.

5 **The list must be sorted; Each look halves what is left; 1,000 items need about 10 looks at most**

Binary search halves a sorted list at every look.